

File	Content	Preprocessor	Compiler - produces object files. Compiler needs the declarations to allocate the placeholders!	Linker - puts together object files and resolves placeholders to addresses
Libraries: stdio.h etc.	Header	Not processed but included in cpp files. Because they are included the compiler knows how to allocate the placeholders.	Libraries (compiled/object files) are available, no compiling needed	/* Executable MyProgInC.exe, detailed in sections */ [/* Compiled C-Libraries like stdio. Linked usually at the beginning */ Address method cout << ... /* more library assembler code */ /* machine code for Person.cpp */ jump to address method cout << "Hello World" Address method getName() ... /* more assembler code */ ... /* more assembler code */ /* machine code for Employee.cpp */ ... string s = jump to Address method Person.getName() ...]

Running a C/C++ program MyProgInC.exe:

On the target machine the MyProgInC.exe runs w/o any additional help.

It uses directly the systems resources, i.e. Operating System (OS).

All libraries needed are already included in MyProgInC.exe (Linker!), it is self-sufficient.

At compile and runtime we have the same code from libraries. The needed functions are packed in the exe file.

Pros: Runs fast. Self-sufficient.

Cons: Runs only on the architecture it is compiled for

Bottom line: "C/C++ pack their own Toothbrush/HashMaps"

Running a Java program:

On the target machine the MyJavaProg.class file is run by the JRE - java runtime(!).

All bindings are provided by the JRE, ex: HashMap class.

At compile and runtime our java program uses different libraries, ex. HashMap.

Exception: The libraries we can pack into the java archive .jar

However, what's the point packing JRE's libraries, ex. HashMap??

Note: Not knowing what are the libraries at runtime has lead to "the jar hell" --> see google.

Java introduced for this the module concept.

Example: 2 libraries offer Log.log(String msg). Which one is used???

Answer: It depends which one is first on the CLASSPATH

Trouble: I want the method only from one JAR, all the others ought to shut up

How to forbid other libraries to provide Log.log(String msg) ???

Answer: **module**

Jar hell awkwardness: With the library environment at compile & test time it works, at runtime the java program crashes :(.

Pros: Runs on every architecture JRE is running. Cons: Slow. Not self-sufficient, "Jar Hell".

Bottom line: "Java does not pack their own Toothbrush/HashMap"